Лабораторная работа №2. Основы Git.

Содержание

Содержание

1 Цели лабораторной работы **4**

2 Задачи к лабораторной работе **4**

3 Методические материалы **5**

3.1 Системы контроля версий 5

3.2 Знакомство с Git 5

3.2.1 Целостность Git 6

3.2.2 Состояния 7

3.2.3 Что внутри? 8

3.3 Ветвление в Git 9

3.3.1 Ветки 9

3.3.2 Теги 13

3.4 Слияние и перебазирование веток 15

3.4.1 Fast-forward 15

3.4.2 Merge 16

3.4.3 Rebase 18

4 Ход работы **21**

4.1 Установка Git 21

4.2 Выполнение работы 21

4.2.1 Создание нового репозитория 21

4.2.2 Работа с репозиторием 22

4.2.3 Создание мастер репозитория 26

4.2.4 Важность root-commit 28
4.2.5 Клонирование репозитория 29
4.2.6 Конфликт и решение конфликта 29

5 Контрольные вопросы **34**

1 Цели лабораторной работы

- Познакомиться с технологией Git и её концепциями;
- Изучить основные команды и возможности git.

2 Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Установить git;
3. Создать новый репозиторий, добавить файлы и создать коммит;
4. Создать и настроить мастер репозиторий, запушить в него изменения;
5. Склонировать мастер репозиторий;
6. Разрешить конфликт версий файла.

3 Методические материалы

3.1 Системы контроля версий

Система контроля версий (СКВ) - система, записывающая изменения в файл или набор файлов в течение времени и позволяющая вернуться позже к определённой версии. Она позволяет вернуть файлы к состоянию, в котором они были до изменений, увидеть сами изменения кто был их автором.

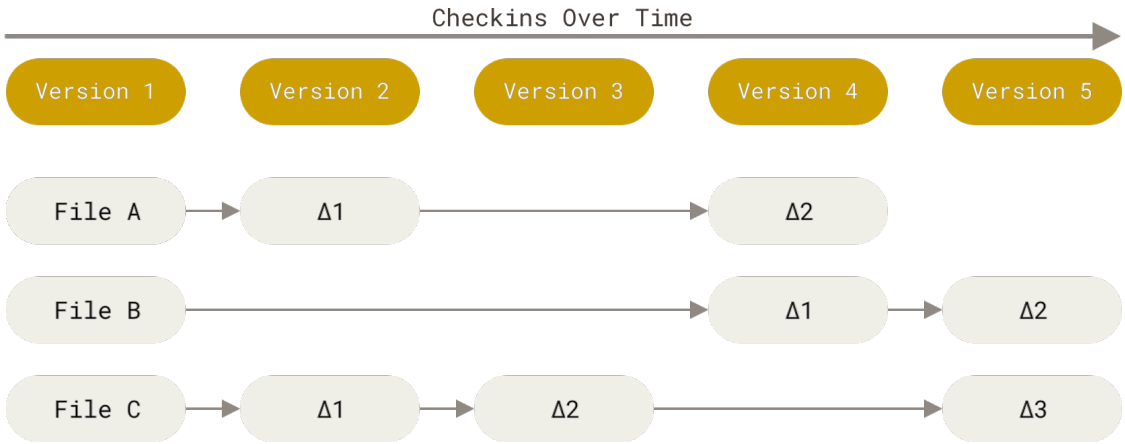
Системы контроля версий можно разделить на локальные, централизованные и распределённые:

- **локальные** – хранят историю изменений и управляют ими на самой машине;
- **централизованные** – появились как способ решения проблемы взаимодействия нескольких разработчиков. На специальном сервере хранятся все версии файлов. К этому серверу обращаются клиенты, которые получают отдельные файлы из централизованного хранилища. Однако у такого метода есть один серьёзный минус – единая точка отказа, представленная централизованным сервером. Если сервер выйдет из строя или жёсткий диск, на котором хранится центральная БД, окажется повреждён, то будет потеряна вся история проекта, за исключением отдельных фиксаций на локальных машинах разработчиков;
- **распределённые** – клиенты не просто скачивают снимок всех файлов - они полностью копируют репозиторий. В таком случае локальная машина каждого разработчика может считаться полным бэкапом проекта на случай, если сервер, через который обмениваются файлами клиенты, выйдет из строя.

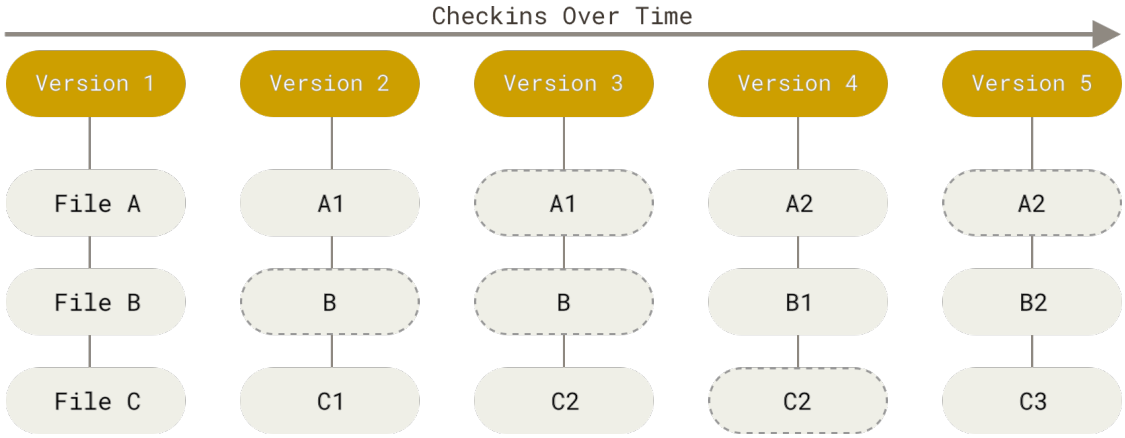
3.2 Знакомство с Git

Git – бесплатная распределенная система контроля версий с открытым исходным кодом. Она была разработана сообществом разработчиков ядра Linux в 2005 году как замена проприетарной СКВ BitKeeper, ранее использовавшейся при разработке ядра Linux.

Основное отличие Git от других систем – подход к работе с данными. Большинство СКВ хранят информацию в виде набора файлов и изменений (дельт), сделанных в каждом файле, по времени (контроль версий, ***основанный на различиях***).



Подход Git больше похож на набор снимков файловой системы: при каждом сохранении состояния проекта в Git система запоминает, как выглядит каждый файл в этот момент, и сохраняет ссылку на этот снимок. Для увеличения эффективности, если файлы не были изменены, Git не запоминает их вновь, а только создаёт ссылку на предыдущую версию идентичного файла, который уже сохранён.



3.2.1 Целостность Git

В Git для всего вычисляется хеш-сумма, и только потом происходит сохранение. В дальнейшем обращение к сохранённым объектам происходит по этой хеш-сумме.

Механизм, которым пользуется Git при вычислении хеш-сумм, называется SHA-1 хеш. Это строка длиной в 40 шестнадцатеричных символов (0-9 и a-f), она вычисляется на основе содержимого файла или структуры каталога. SHA-1 хеш выглядит примерно так:

```
24b9da6552252987aa493b52f8696cd6d3b00373
```

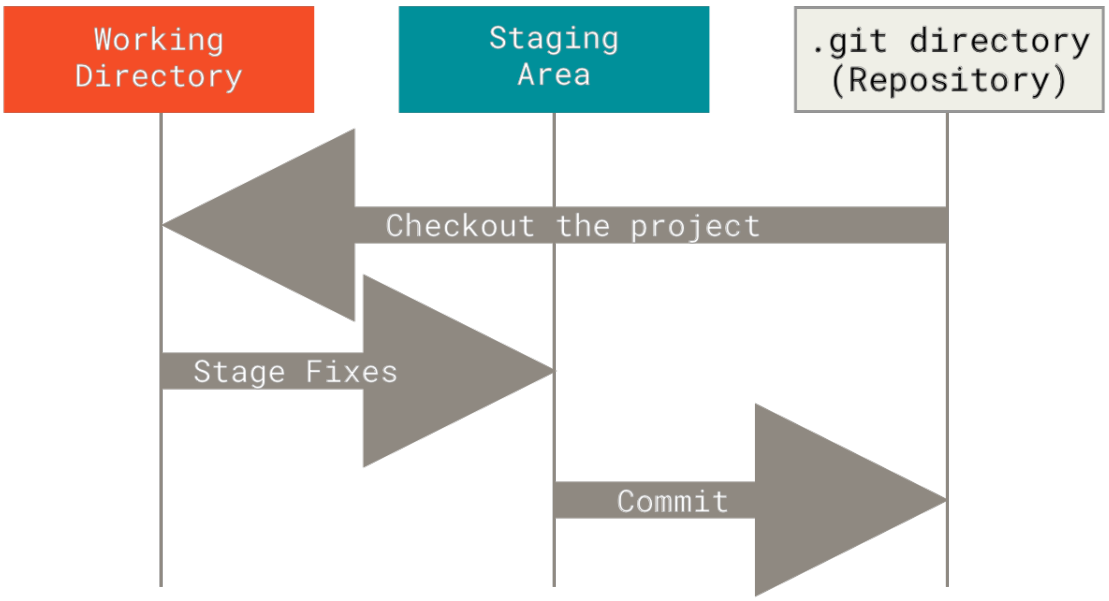
Git постоянно использует хеши при работе с репозиторием.

3.2.2 Состояния

У Git есть три основных состояния, в которых могут находиться файлы: **изменён** (modified), **индексирован** (staged) и **зафиксирован** (committed):

- К изменённым относятся файлы, которые поменялись, но ещё не были зафиксированы.
- Индексированный – это изменённый файл в его текущей версии, отмеченный для включения в следующий коммит.
- Зафиксированный значит, что файл уже сохранён в локальной базе.

Также Git имеет три основные секции проекта: рабочая копия (working tree), область индексирования (staging area, индекс) и каталог Git (Git directory):

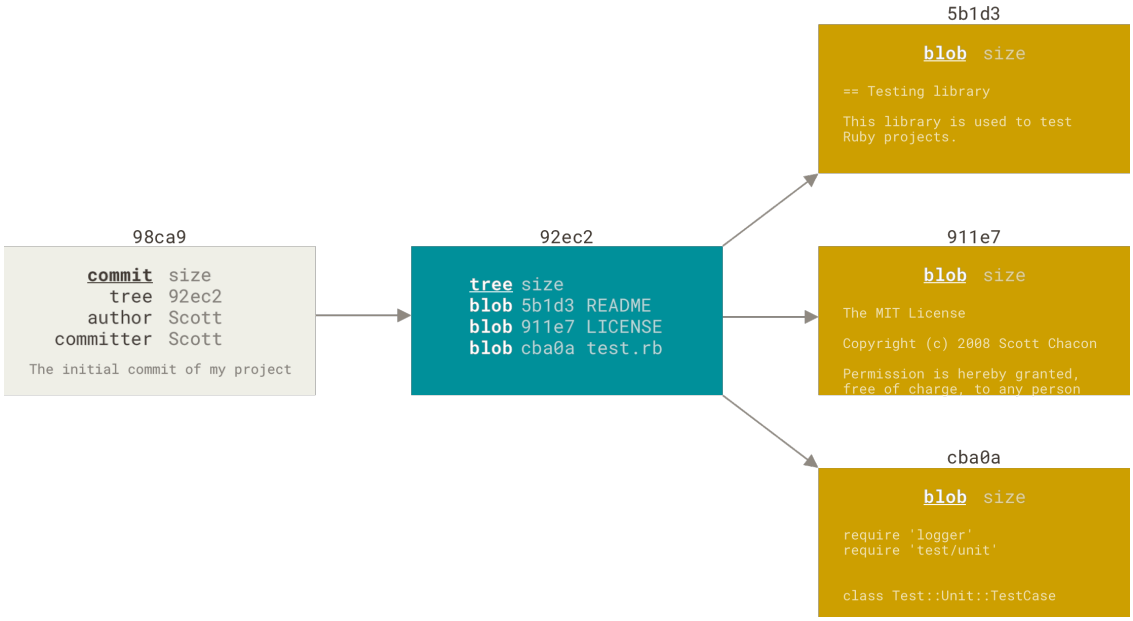


- Рабочая копия является снимком одной версии проекта. Эти файлы извлекаются из сжатой базы данных в каталоге Git и помещаются на диск, для того чтобы их можно было использовать или редактировать.
- Область индексирования – это файл, обычно находящийся в каталоге Git, в нём содержится информация о том, что попадёт в следующий коммит.
- Каталог Git – это то место, где Git хранит метаданные и базу объектов вашего проекта. Это самая важная часть Git и это та часть, которая копируется при клонировании репозитория с другого компьютера.

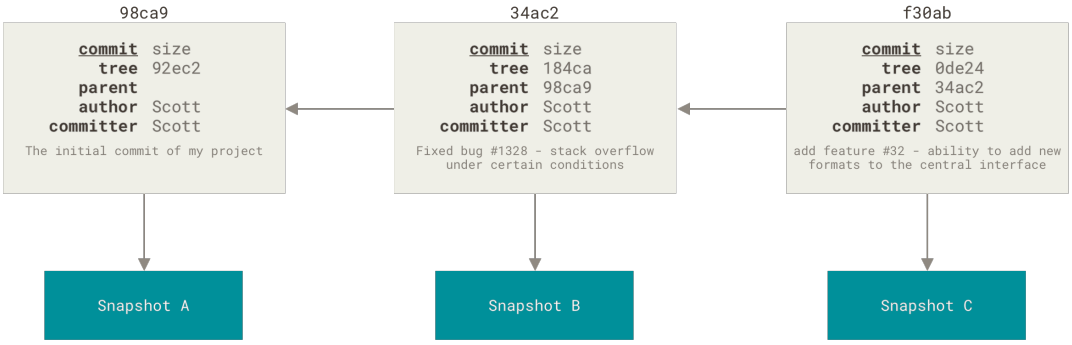
3.2.3 Что внутри?

Git состоит из трёх структур: **blob** (большой бинарный объект), **tree** (деревья), **commit** (коммиты):

- В объекте типа **blob** хранится содержимое одного конкретного файла.
- **tree** – это файловая структура проекта, в которой содержится список ссылок на объекты **blob** и список ссылок на дочерние деревья (вложенные каталоги).
- В объекте **commit** хранится ссылка на объект **tree** и ссылка на родительский коммит.



В свою очередь коммиты объединяются в однонаправленный список от более новых к более старым, то есть в каждом коммите, кроме первого, есть ссылка на родительский коммит.



3.3 Ветвление в Git

Механизм ветвления в системах контроля версий позволяет вести независимо параллельную разработку, поддерживая одновременно несколько версий проекта.

3.3.1 Ветки

Ветка в Git – это перемещаемый указатель коммит. По умолчанию, имя основной ветки в Git – `master`. `HEAD` – указатель на текущую ветку или коммит, на котором мы в данный момент находимся.

Разберем на примере.

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git init
Initialized empty Git repository in /home/pndsdn/learnGit/.git/
pndsdn@pndsdn:~/learnGit$ echo "hello, world" > readme.md
pndsdn@pndsdn:~/learnGit$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        readme.md

nothing added to commit but untracked files present (use "git add" to track)
pndsdn@pndsdn:~/learnGit$ git add readme.md
pndsdn@pndsdn:~/learnGit$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.md

pndsdn@pndsdn:~/learnGit$ git commit -m "init commit"
[main (root-commit) 9703228] init commit
 1 file changed, 1 insertion(+)
 create mode 100644 readme.md
pndsdn@pndsdn:~/learnGit$ git log
commit 970322862aa589e2c24882c05052639b163c630c (HEAD -> main)
Author: Andrey Sopov <saa020403@gmail.com>
Date:   Fri Feb 28 01:23:39 2025 +0300

    init commit
pndsdn@pndsdn:~/learnGit$
```

На рисунке выше:

1. Создан локальный репозиторий (`git init`);

2. Файл `readme.md` с содержимым "hello, world";

3. Была получена информация о текущем состоянии репозитория (`git status`). Вывод показывает, что в рабочей копии находится файл `readme.md`;

4. Добавляем файл в индекс (`git add readme.md`), тем самым говорим, что хотим зафиксировать его изменения;

5. Проверяем статус ещё раз и видим, что теперь файл помечен как готовый к фиксации (коммиту);

6. Коммитим изменения с сообщением "init commit"(`git commit -m 'init commit'`). Если не указать флаг `-m` с сообщением, то откроется редактор по умолчанию (к примеру, vim или nano) и будет предложено ввести комментарий;

7. Последним действием проверяем историю коммитов (`git log`). Сейчас в ней находится только что созданный коммит.

Добавим файл `main.py`, закоммитим и посмтотрим на историю:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ echo "print('hello, python')" > main.py
pndsdn@pndsdn:~/learnGit$ git add main.py
pndsdn@pndsdn:~/learnGit$ git commit -m "add main.py"
[main e7b2476] add main.py
 1 file changed, 1 insertion(+)
 create mode 100644 main.py
pndsdn@pndsdn:~/learnGit$ git log
commit e7b2476821fa6b44235dab0a4022f62be2a4d64d (HEAD -> main)
Author: Andrey Sopov <saa020403@gmail.com>
Date:   Fri Feb 28 11:01:07 2025 +0300

    add main.py

commit 970322862aa589e2c24882c05052639b163c630c
Author: Andrey Sopov <saa020403@gmail.com>
Date:   Fri Feb 28 01:23:39 2025 +0300

    init commit
pndsdn@pndsdn:~/learnGit$
```

Обратите внимание, что указатель на ветку `main` переместился на последний коммит. `HEAD` указывает на ветку `main` (`HEAD -> main`).

Рассмотрим процесс создания, переключения и удаления веток:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git branch
* main
pndsdn@pndsdn:~/learnGit$ git branch feature
pndsdn@pndsdn:~/learnGit$ git branch
  feature
* main
pndsdn@pndsdn:~/learnGit$ git checkout feature
Switched to branch 'feature'
pndsdn@pndsdn:~/learnGit$ git branch
* feature
  main
pndsdn@pndsdn:~/learnGit$ git checkout -b another_feature
Switched to a new branch 'another_feature'
pndsdn@pndsdn:~/learnGit$ git branch
* another_feature
  feature
  main
pndsdn@pndsdn:~/learnGit$ git checkout -b feature
fatal: a branch named 'feature' already exists
pndsdn@pndsdn:~/learnGit$ git branch -d another_feature
error: cannot delete branch 'another_feature' used by worktree at '/home/pndsdn/learnGit'
pndsdn@pndsdn:~/learnGit$ git checkout main
Switched to branch 'main'
pndsdn@pndsdn:~/learnGit$ git branch -d another_feature
Deleted branch another_feature (was e7b2476).
pndsdn@pndsdn:~/learnGit$ git branch
  feature
* main
pndsdn@pndsdn:~/learnGit$
```

1. Вывели список существующих веток (`git branch`). На данный момент у нас есть только одна ветка `main`. Зелёный шрифт и знак `*` означают указатель `HEAD`;

2. Создали новую ветку `feature` (`git branch feature`);

3. Обратите внимание, что `HEAD` по прежнему указывает на `main`. Если на этой стадии сделать коммит, то вперёд продвинется указатель `main`, а не `feature`;

4. Переключились на ветку `feature` командой `git checkout feature`;

5. Теперь вывод списка сообщает, что указатель `HEAD` переместился на ветку `feature`;

6. С помощью команды `git checkout -b another_feature` создается ветка `another_feature` и указатель `HEAD` перемещается на неё;

7. Однако вызов команды `git checkout -b feature` приводит к ошибке, так как ветка с таким именем уже существует;

8. Удаление ветки делается с помощью флага `-d` команды `git branch`. Но попытка удалить ветку, на которой вы находитесь, приведёт к ошибке. Поэтому

сначала переключаемся на другую ветку, а только после этого удаляем.

Как говорилось ранее, ветки позволяют поддерживать несколько версий проекта. Продемонстрируем это на примере:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* e7b2476 (HEAD -> main, feature) add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ echo "hello from main" > readme.md
pndsdn@pndsdn:~/learnGit$ git add readme.md
pndsdn@pndsdn:~/learnGit$ git commit -m main
[main 5694728] main
 1 file changed, 1 insertion(+), 1 deletion(-)
pndsdn@pndsdn:~/learnGit$ git checkout feature
Switched to branch 'feature'
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 5694728 (main) main
* e7b2476 (HEAD -> feature) add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ echo "hello from feature" > readme.md
pndsdn@pndsdn:~/learnGit$ git add readme.md
pndsdn@pndsdn:~/learnGit$ git commit -m feature
[feature eb1fb68] feature
 1 file changed, 1 insertion(+), 1 deletion(-)
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* eb1fb68 (HEAD -> feature) feature
| * 5694728 (main) main
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ cat readme.md
hello from feature
pndsdn@pndsdn:~/learnGit$ git checkout main
Switched to branch 'main'
pndsdn@pndsdn:~/learnGit$ cat readme.md
hello from main
pndsdn@pndsdn:~/learnGit$
```

1. Команда `git log --all --decorate --oneline --graph` выводит историю коммитов в виде графа всех веток с коротким коментарием в одну строку;
2. В ветке `main` обновим файл `readme.md` и зафиксируем изменения;
3. Переключимся на ветку `feature`. Обратите внимание, что она указывает не на последний коммит, а на тот, на котором она находилась до этого (хеш `e7b2476`);
4. Опять поменяем содержимое `readme.md` и закоммитим;
5. Теперь мы видим, как наш проект разделился на две независимые ветки, с общим родителем - коммитом `e7b2476`;
6. Проверили, что файл `readme.md` имеет две версии с изменениями в соответствующих ветках.

3.3.2 Теги

Теги в Git - это способ помечать определённые коммиты. Как правило они используются для отметок версий приложения (`v1.0.0` и т.п.)

Git поддерживает два вида тегов:

- легковесные – указатель на определенный коммит, который не изменяется. По сути, это контрольная сумма коммита, сохранённая в файл;
- аннотированные – это полноценные объекты, которые имеют контрольную сумму, имя автора, его e-mail, дату создания и комментарий. Обычно рекомендуется создавать аннотированные теги.

Создадим два тега: легковесный и аннотированный:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* eb1fb68 (feature) feature
| * 5694728 (HEAD -> main) main
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git tag v1.0-lw
pndsdn@pndsdn:~/learnGit$ git tag -a v1.0 -m "release v1.0"
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* eb1fb68 (feature) feature
| * 5694728 (HEAD -> main, tag: v1.0-lw, tag: v1.0) main
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$
```

1. Для создания легковесного тега надо вызвать команду `git tag <tag_name>` без всяких параметров;
2. Для создания аннотированного тега необходимо передать параметр `-a`. Если не указать параметр `-m` с сообщением, то откроется текстовый редактор с предложением написать комментарий.

Сравним результаты вывода информации о тегах:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git show v1.0-lw
commit 5694728b469033cf66b2aae3a3c5275d0a29cea9 (HEAD -> main, tag:
v1.0-lw, tag: v1.0)
Author: Andrey Sopov <saa020403@gmail.com>
Date:   Fri Feb 28 12:25:50 2025 +0300

    main

diff --git a/readme.md b/readme.md
index 4b5fa63..d42034c 100644
--- a/readme.md
+++ b/readme.md
@@ -1 +1 @@
-hello, world
+hello from main
pndsdn@pndsdn:~/learnGit$
```

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git show v1.0
tag v1.0
Tagger: Andrey Sopov <saa020403@gmail.com>
Date:   Fri Feb 28 17:02:06 2025 +0300

release v1.0

commit 5694728b469033cf66b2aae3a3c5275d0a29cea9 (HEAD -> main, tag:
v1.0-lw, tag: v1.0)
Author: Andrey Sopov <saa020403@gmail.com>
Date:   Fri Feb 28 12:25:50 2025 +0300

    main

diff --git a/readme.md b/readme.md
index 4b5fa63..d42034c 100644
--- a/readme.md
+++ b/readme.md
@@ -1 +1 @@
-hello, world
+hello from main
pndsdn@pndsdn:~/learnGit$
```

1. В первом случае видно, что тег просто ссылается на коммит, никакой дополнительной информации мы не видим;

2. Во втором случае, помимо информации о коммите, видно кто и когда создал тег, а также комментарий к нему.

3.4 Слияние и перебазирование веток

В процессе работы у вас может появится потребность объединить код из разных веток: к примеру, в основную ветку вашего проекта надо добавить функционал из ветки `feature`. Git предоставляет несколько способов, как это можно сделать.

3.4.1 Fast-forward

В приведённом ниже примере ветка `feature` опережает ветку `main` на два коммита, то есть ветка `main` не имеет параллельных версий, а граф имеет линейную структуру. В такой ситуации достаточно перенести указатель вперёд (fast-forward):

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 046a91f (HEAD -> feature) update readme
* eb1fb68 feature
* e7b2476 (main) add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git switch main
Switched to branch 'main'
pndsdn@pndsdn:~/learnGit$ git merge feature
Updating e7b2476..046a91f
Fast-forward
 readme.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 046a91f (HEAD -> main, feature) update readme
* eb1fb68 feature
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$
```

1. Для слияния ветки `<source_branch>` в ветку `<target_branch>` с помощью команды `git merge <source_branch>` необходимо находится на `<target_branch>`. Так как мы хотим влить ветку `feature` в `main`, нам надо переключится на ветку `main`;

2. Выполним команду `git merge feature`. Обратите внимание на сообщение "Fast-forward"в выводе;

В случаях, когда при fast-forward слиянии необходимо оставить историю, можно к команде `git merge <source_branch>` добавить флаг `--no-ff` (no fast-forward), тогда будет создан новый коммит:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 046a91f (feature) update readme
* eb1fb68 feature
* e7b2476 (HEAD -> main) add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git merge feature --no-ff
Merge made by the 'ort' strategy.
 readme.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
*   e8dbce6 (HEAD -> main) Merge branch 'feature'
|\
| * 046a91f (feature) update readme
| * eb1fb68 feature
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$
```

3.4.2 Merge

В ситуации, когда коммит, на котором мы находимся, не является прямым родителем ветки, с которой мы выполняем слияние, Git выполняет простое трёхстороннее слияние, используя последние коммиты объединяемых веток и общего для них родительского коммита.

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 4e80d91 (HEAD -> main) add one more file
| * 046a91f (feature) update readme
| * eb1fb68 feature
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git merge feature
Merge made by the 'ort' strategy.
 readme.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
*   70fee89 (HEAD -> main) Merge branch 'feature'
|\
| * 046a91f (feature) update readme
| * eb1fb68 feature
* | 4e80d91 add one more file
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$
```

В данном примере в ветку `main` был добавлен новый файл, которого нет в ветке

`feature`. Следовательно, при слиянии Git объединил версии `feature` и `main` в один новый коммит.

Но в процессе разработки, один и тот же файл может независимо изменяться в разных ветках, и тогда при слиянии появится конфликт версий, который нам придется решить вручную, и после закончить слияние:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 565a561 (HEAD -> main) change greeting text
| * 046a91f (feature) update readme
| * eb1fb68 feature
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git merge feature
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
Automatic merge failed; fix conflicts and then commit the result.
pndsdn@pndsdn:~/learnGit$ cat readme.md
<<<<<<< HEAD
hello and welcome to los pollos hermanos family
=======
# Greeting
Hello, World!
>>>>>>> feature
pndsdn@pndsdn:~/learnGit$ vim readme.md
pndsdn@pndsdn:~/learnGit$ cat readme.md
# Greeting
hello and welcome to los pollos hermanos family
pndsdn@pndsdn:~/learnGit$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   readme.md

no changes added to commit (use "git add" and/or "git commit -a")
pndsdn@pndsdn:~/learnGit$ git add readme.md
pndsdn@pndsdn:~/learnGit$ git commit
[main 2ca2e66] Merge branch 'feature'
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
*   2ca2e66 (HEAD -> main) Merge branch 'feature'
|\
| * 046a91f (feature) update readme
| * eb1fb68 feature
* | 565a561 change greeting text
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$
```

В примере выше файл `readme.md` был изменён в двух ветках. При попытке их

слияния Git выдал сообщение о конфликте. При этом файл внутри себя содержит информацию о изменениях текущей ветки (строки между `<<<<<<< HEAD` и `=======`) и вливаемой ветки (строки между `=======` и `>>>>>>> feature`).

Предположим, что мы хотим оставить строку `# Greeting`, но поменять приветствие на то, которое было сделано в ветке `main`. Отредактируем файл соответствующим образом.

При проверке статуса, Git сообщает, что имеются неслитые изменения. Добавляем изменённый файл в индекс и коммитим.

3.4.3 Rebase

Суть метода rebase (перебазирования) заключается в том, чтобы взять изменения из одной ветки и применить их поверх другой. В таком случае берётся общий родительский коммит двух веток (текущей, и той, поверх которой вы выполняете перебазирование), определяется дельта каждого коммита текущей ветки и сохраняется во временный файл, текущая ветка устанавливается на последний коммит ветки, поверх которой вы выполняете перебазирование, а затем по очереди применяются дельты из временных файлов.

В отличие от команды `git merge`, для `git rabase` необходимо находиться на той ветке, которая будет перебазирована.

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 565a561 (main) change greeting text
| * 046a91f (HEAD -> feature) update readme
| * eb1fb68 feature
|/
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git rebase main
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
error: could not apply eb1fb68... feature
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue
".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git
 rebase --abort".
Could not apply eb1fb68... feature
pndsdn@pndsdn:~/learnGit$ cat readme.md
<<<<<<< HEAD
hello and welcome to los pollos hermanos family
=======
hello from feature
>>>>>>> eb1fb68 (feature)
pndsdn@pndsdn:~/learnGit$ vim readme.md
pndsdn@pndsdn:~/learnGit$ cat readme.md
hello and welcome to los pollos hermanos family
pndsdn@pndsdn:~/learnGit$ git add readme.md
pndsdn@pndsdn:~/learnGit$ git rebase --continue
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
error: could not apply 046a91f... update readme
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue
".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git
 rebase --abort".
Could not apply 046a91f... update readme
pndsdn@pndsdn:~/learnGit$
```

При перебазировании ветки у нас возникает конфликт. Смотрим содержимое и видим, что это конфликт с коммитом `eb1fb68` – всё верно, так как было сказано ранее: при перебазировании определяются изменения для каждого коммита ветки. Решаем конфликт и продолжаем процедуру (`git rebase --continue`) и снова встречаем конфликт, но уже коммита `046a91f`:

```
pndsdn@pndsdn: ~/learnGit
pndsdn@pndsdn:~/learnGit$ cat readme.md
<<<<<<< HEAD
hello and welcome to los pollos hermanos family
=======
# Greeting
Hello, World!
>>>>>>> 046a91f (update readme)
pndsdn@pndsdn:~/learnGit$ vim readme.md
pndsdn@pndsdn:~/learnGit$ git add readme.md
pndsdn@pndsdn:~/learnGit$ git rebase --continue
[detached HEAD 05b867c] update readme
 1 file changed, 1 insertion(+)
Successfully rebased and updated refs/heads/feature.
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 05b867c (HEAD -> feature) update readme
* 565a561 (main) change greeting text
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$ git checkout main
Switched to branch 'main'
pndsdn@pndsdn:~/learnGit$ git merge feature
Updating 565a561..05b867c
Fast-forward
 readme.md | 1 +
 1 file changed, 1 insertion(+)
pndsdn@pndsdn:~/learnGit$ git log --all --decorate --oneline --graph
* 05b867c (HEAD -> main, feature) update readme
* 565a561 change greeting text
* e7b2476 add main.py
* 9703228 init commit
pndsdn@pndsdn:~/learnGit$
```

Решаем конфликт и продолжаем перебазирование (`git rebase --continue`). После завершения переноса коммитов граф принимает линейный вид. Теперь мы можем переключится на ветку `main` и выполнить быстрое перемещение указателя.

Обратите внимание, что "перенесённые"коммиты имеют отличный от оригинала хеш. Всё по тому, что это новые коммиты, а исходные попали в состояние, когда ни одна ветка не указывает на них.

4 Ход работы

4.1 Установка Git

ПРИМЕЧАНИЕ: Данную лабораторную работу можно выполнять в любой из трёх ОС:

- Windows
- Linux
- Mac

1. Перейдите на страницу загрузки Git;
2. Выберите вариант установки для вашей операционной системы;
3. Установите Git для всех пользователей;
4. Если операционная система Windows:
   - Откройте через поиск **Git Bash**;
   - Перейдите в папку лабораторной работы №2;

   Если операционная система Linux или Mac:
   - Откройте терминал в папке лабораторной работы №2;

4.2 Выполнение работы

4.2.1 Создание нового репозитория

Создадим папку для репозитория git:

```
mkdir repo_new
```

На данный момент `repo_new` является просто папкой, для того, чтобы получить репозиторий, выполним следующие команды:

```
cd repo_new
git init
```

Мы должны увидеть в консоли:

```
Initialized empty Git repository in Путь< к лабораторной работе №2>/
   repo_new/.git/
```

Теперь `repo_new` является частично инициализированным git репозиторием. Папка `.git` содержит файлы, необходимые для работы git, и саму историю версий. Всё остальное пространство называется ***working tree*** (рабочее дерево), в этом пространстве мы можем работать с нашими файлами: создавать, редактировать, удалять.

ПРИМЕЧАНИЕ: Используйте команды `ls`, `ls -a` и `ls -la` для изучения содержимого папок `repo_new` и `.git`. Вы можете заметить, что без флага `-a` папка `.git` не отображается.

Причина этому соглашение об именах в Unix подобных системах: файлы и папки, чьи имена начитаются с символа `.` считаются скрытыми.

Теперь выполним:

```
git status
```

Вы должны получить:

```
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Где:

- Первая строка сообщает нам текущую ветку master (это базовая ветка, являющаяся основой нашего репозитория);
- Вторая строка сообщает, что у нас отсутствует root коммит (частично инициализированный репозиторий);
- Третья строка сообщает, что у нас нет файлов, которые мы могли бы добавить в историю версий (закоммитить), а также предлагает нам это сделать.

4.2.2 Работа с репозиторием

Создадим новый пустой файл:

```
touch file.txt
```

Выполним `git status` и посмотрим на изменения:

```
On branch master

No commits yet

Untracked files:
    (use "git add <file>..." to include in what will be committed)
        file.txt

nothing added to commit but untracked files present (use "git add" to
   track)
```

Появилась строка, сообщающая о не отслеживаемых файлах (`file.txt`) с предложением этот файл отслеживать. А также изменилось сообщение в последней строке.

Включим отслеживание для всех файлов (если в команду `git add` передать `.` вместо имени файла, будут отслеживаться все файлы в рабочем каталоге).

```
git add .
```

Снова проверяем статус:

```
On branch master

No commits yet

Changes to be committed:
    (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
```

Сообщение снова изменилось. Теперь у нас есть отслеживаемые файлы, которые мы можем добавить в коммит. Если же мы ошиблись, и добавили лишний файл, то всегда можем сделать `git rm --cached <file>`. Файлы, что не входят в текущий коммит, называются *unstaged*.

ПРИМЕЧАНИЕ: Флаг `--cached` позволяет отключить отслеживание файла, не удаляя его физически. Без этого флага файл будет удалён с диска **безвозвратно**.

Важно понимать, что git работает только с ***файлами***. Если необходимо отслеживать структуру директорий, необходимо пойти на определённую хитрость.

Создадим пустой каталог:

```
mkdir empty
```

И проверим статус репозитория:

```
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
```

Статус не изменился. Наша директория не отслеживается.

Исправим это:

```
cd empty
touch .gitkeep
cd ..
```

Мы создали специальный пустой файл, который негласно принято использовать для удержания пустых директорий: `.gitkeep`.

Статус репозитория:

```
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        empty/
```

Повторяем `git add .`:

```
On branch master

No commits yet
```

```
Changes to be committed:
    (use "git rm --cached <file>..." to unstage)
        new file:   empty/.gitkeep
        new file:   file.txt
```

Теперь мы отслеживаем `empty/.gitkeep` файл и сохраняем информацию о директории `empty`.

Последнее, что нам необходимо сделать, это зафиксировать (закоммитить) наши изменения:

```
git commit -m "First commit"
```

Результат операции включает:

- Ветку, в которой зафиксированы изменения (master);
- Флаг, что это первый, корневой, коммит;
- Первые шесть символов хеша коммита;
- Сообщение коммита;
- Количество изменённых файлов, количество добавленных и удалённых строк;
- Список файлов.

```
[master (root-commit) b95ba56] First commit
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 empty/.gitkeep
 create mode 100644 file.txt
```

ПРИМЕЧАНИЕ: Команда `git commit` требует, чтобы заранее были установлены имя пользователя и почта автора, от лица которого осуществляется коммит. Команды для установки:

```
git config user.name "User name"
git config user.email "user@email.com"
```

Если необходимо установить имя пользователя и почту глобально, используйте флаг `--global` после `config`.

Статус репозитория после коммита:

```
On branch master
nothing to commit, working tree clean
```

Фраза *"working tree clean"* говорит о том, что в рабочем пространстве все файлы соответствуют зафиксированному состоянию.

4.2.3 Создание мастер репозитория

Вернёмся в корневой каталог лабораторной работы:

```
cd ..
```

Создадим мастер репозиторий:

```
mkdir repo_master
cd repo_master
git init
```

Вывод:

```
Initialized empty Git repository in Путь< к лабораторной работе №2>/
   repo_master/.git/
```

Вернёмся в `repo_new` и попробуем отправить изменения в созданный мастер репозиторий:

```
cd ../repo_new
```

Теперь нам необходимо выполнить несколько шагов для связи нашего `repo_new` с `repo_master`.

Настраиваем ссылку на удалённый репозиторий:

```
git remote add origin "../repo_master"
```

ПРИМЕЧАНИЕ: Имя удалённого репозитория *origin* принято как стандартное имя для главного удалённого репозитория.

Для первой операции `git push` необходимо указать соответствующую удалённую ветку:

```
git push --set-upstream origin master
```

Однако выполнение данной команды вернёт ошибку:

```
...
```

```
remote: error: refusing to update checked out branch: refs/heads/master
remote: error: By default, updating the current branch in a non-bare
   repository
remote: is denied, because it will make the index and work tree
   inconsistent
remote: with what you pushed, and will require 'git reset --hard' to
   match
remote: the work tree to HEAD.

...

To ../repo_master
 ! [remote rejected] master -> master (branch is currently checked out)
error: failed to push some refs to '../repo_master'
```

Причина – неверно проинициализированный мастер репозиторий. Исправим его:

```
cd ..
rm -rf repo_master
mkdir repo_master
cd repo_master
git init --bare
```

Вывод:

```
Initialized empty Git repository in Путь< к лабораторной работе №2>/
   repo_master/
```

Мы используем флаг `--bare` для создания специального репозитория. В данном репозитории отсутствует рабочее пространство и папка `.git`. И данный репозиторий разрешает пушить root-commit, если его ещё нет.

Вернёмся в `repo_new` и выполним push:

```
cd ../repo_new
git push --set-upstream origin master
```

Вывод:

```
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 297 bytes | 297.00 KiB/s, done.
```

```
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To ../repo_master
    * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Часть вывода ** [new branch]* сообщает, что в удалённом репозитории была создана ветка *master*.

Повторная команда `git push` выдаст сообщение о том, что всё синхронизировано:

```
Everything up-to-date
```

Статус репозитория:

```
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
```

4.2.4 Важность root-commit

Создадим новый репозиторий `repo_test`:

```
cd ..
mkdir repo_test
cd repo_test
git init
```

Создадим и закоммитим файл:

```
touch rand.txt
git add .
git commit -m "aaa"
```

И попробуем отправить этот коммит в мастер репозиторий:

```
git remote add origin "../repo_master"
git push --set-upstream origin master
```

Получаем ошибку:

```
To ../repo_master
! [rejected]        master -> master (fetch first)
error: failed to push some refs to '../repo_master'
```

```
hint: Updates were rejected because the remote contains work that you
   do not
hint: have locally. This is usually caused by another repository
   pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for
   details.
```

Причина: root-commit в `repo_test` и `repo_master` различаются.

Удалим не нужный больше `repo_test`:

```
cd ..
rm -rf repo_test
```

4.2.5 Клонирование репозитория

Для клонирования используется команда `git clone <URL> [<local_name>]`. Выполним:

```
git clone "./repo_master" "repo_cloned"
cd repo_cloned
git status
```

Статус аналогичен `repo_new`:

```
On branch master
Your branch is up to date with 'origin/master'.
```

4.2.6 Конфликт и решение конфликта

Изменим содержимое файла `file.txt`:

```
nano file.txt
git status
```

Статус обновился и появилась новая информация:

```
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

```
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Теперь git предлагает два новых варианта:

- `git restore ...` - Для отката изменений файла к зафиксированной версии;
- `git commit -a` - Для автоматического включения всех отслеживаемых изменений в коммит.

Сделаем коммит:

```
git add .
git commit -m "second"
git push
```

Перейдём обратно в `repo_new`:

```
cd ../repo_new
```

Теперь изменим файл `file.txt` и в этом репозитории, но введём другое содержимое:

```
nano file.txt
git add .
git commit -m "other second"
```

И попробуем сделать push. Мы получим ошибку, так как версии наша и в мастер репозитории различаются:

```
git push
```

Вывод:

```
To ../repo_master
! [rejected]        master -> master (fetch first)
error: failed to push some refs to '../repo_master'
hint: Updates were rejected because the remote contains work that you
   do not
hint: have locally. This is usually caused by another repository
   pushing to
hint: the same ref. If you want to integrate the remote changes, use
```

```
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for
   details.
```

Выполним `git pull` и получим изменения из удалённого репозитория:

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 261 bytes | 32.00 KiB/s, done.
From ../repo_master
   b95ba56..b3d64d7  master     -> origin/master
Auto-merging file.txt
CONFLICT (content): Merge conflict in file.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Мы получили информацию о конфликте и файле, который конфликт содержит. Теперь нам необходимо его решить, откроем конфликтный файл:

```
nano file.txt
```

Содержимое файла:

```
<<<<<<< HEAD
<Text from repo_new repository>
=======
<Text from repo_cloned repository>
>>>>>>> b3d64d774c15db0cd026c39ca6d05af59aeae5e8
```

Конструкции из стрелочек и знаков равно выделяют каждый конфликт в файле, который требует решения. Текст после стрелочек указывает, какому коммиту и из какого репозитория содержимое файла.

Для решения необходимо удалить данные управляющие конструкции и написать финальный вариант, который должен быть. После завершения редактирования проверим статус:

```
On branch master
Your branch and 'origin/master' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours
   )
```

```
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   file.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Конфликт всё ещё не решён. Для подтверждения решения конфликта необходимо выполнить `git add file.txt`, тогда статус примет следующий вид:

```
On branch master
Your branch and 'origin/master' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours
   )

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   file.txt
```

Зафиксируем слияние:

```
git commit -m "Merged"
git status
git push
```

Вывод:

```
...

On branch master
Your branch is ahead of 'origin/master' by 2 commits.
    (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

```
...

Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 573 bytes | 573.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To ../repo_master
    b3d64d7..3164901  master -> master
```

Выполним `git log`, чтобы увидеть историю коммитов. Обратите внимание, что последний (самый верхний) коммит имеет дополнительную секцию "Merge". Данная секция показывает нам, что это коммит слияния изменений:

```
commit 3164901f30ed23961c08ac3cf60a93a144b39180 (HEAD -> master, origin
   /master, origin/HEAD)
Merge: 79630b8 b3d64d7
Author: Username <you@email.com>
Date:   Wed Feb 26 20:47:38 2025 +0300

    Merged

commit 79630b80c7b2138eb6cf0a3a70403747d4cefa0e
Author: Username <you@email.com>
Date:   Wed Feb 26 20:35:58 2025 +0300

    other second

commit b3d64d774c15db0cd026c39ca6d05af59aeae5e8
Author: Username <you@email.com>
Date:   Wed Feb 26 20:35:01 2025 +0300

    second

commit b95ba5655ca17c53a162fd94e4b683906167e83b
Author: Username <you@email.com>
Date:   Wed Feb 26 20:51:24 2025 +0300

    First commit
```

5 Контрольные вопросы

1. Что такое Git?

2. Что такое ветка?

3. Объясните метод объединения веток fast-forward.

4. Объясните метод объединения веток merge.